QMFS Senior Team Software Application Task

Ideal Brake Curve Report

Name: Dawud Irfan Rahman

Student ID: *******

Email: ****************

Contents

Objectives.....3

Software Implementations.....3

Theory and Inputs.....3

Table of constants.....3

Initial Load.....4

During Braking.....4

Calculating final front and rear brake forces.....4

Diagonal reference lines.....5

Assumptions and decisions.....6

Results and verification.....7

How to use.....9

Running instruction.....9

References.....9

Source Code.....10

Objectives

The aim of this task is to use python to calculate and plot the ideal braking curve for a vehicle with details of mass, height, centre of gravity height, wheelbase and axle weight distribution given by the document. The graph presents rear axle braking force (y axis) over front axle braking force (x axis), showing the force distributions at which both axles reach their road grip limits simultaneously. It also has labelled reference lines mathematically calculated to identify each friction coefficients.

Software Implementation

I have decided to use python for this task. The reason is that there are two packages I have been introduced to in my IT classes for the module "computational and mathematical modelling 1", those being *Numpy* and *matplotlib*. I have already started self-teaching the basics of these packages and see they have the functions and resources to iterate through sequences of friction coefficients (as decimals), calculate plot points and physically plot calculated forces onto a grid axis. I have used python previously in a levels and GCSE, so the commenting and basic functions and sequence, selection and iteration constructs of programming are all familiar to me. Hence, this is suitable for performing the numerical calculations and display results graphically.

Theory and inputs

Table of constants

Constant	Value / units
GRAVITY_ACCELERATION	9.81 / m s^-2
MASS	280 / Kg
COG_HEIGHT	0.25 / m
WHEELBASE	1.59 / m
FRONT_WEIGHT_DISTRIBUTION	0.5 / no unit

These are the vehicle details provided in the document used to calculate the brake curve.

Initial load

Initially, the total normal reaction load equals the weight and the front load and rear load both share that normal reaction, according to the distribution. The distribution given in the document was 50/50 (assumed as front weight is 0.5).

```
#define and reset front and rear loads
FRONT_LOAD = MASS * GRAVITY_ACCELERATION * FRONT_WEIGHT_DISTRIBUTION
REAR_LOAD = (MASS * GRAVITY_ACCELERATION) * (1 - FRONT_WEIGHT_DISTRIBUTION)
```

In the loop with each value of friction coefficient, I calculated the front and rear load using the code above. It calculates Load using F = m x g x (front axle distribution) for the front load and F = m x g x (1 – front axle distribution) for the rear load. I adjusted the distributions accordingly to calculate front and rear loads (with the values given they are equal).

During Braking

Braking increases front reaction load and decreases the rear reaction load as it transfers to the front. This is a load transfer so mass does not move between axles, only force does.

```
#Deceleration calculation  a = ug
DECELERATION = FRICTION_COEFFICIENT * GRAVITY_ACCELERATION

#Transferred load from rear to front calculation
LOAD_TRANSFER = (MASS * DECELERATION * COG_HEIGHT) / WHEELBASE
```

This code calculates the deceleration using the relationship a = ug, then calculates load transfer during braking using the given formula in the document:

$$\Delta W = \vec{a}m\frac{h_{cg}}{d_{wheel}}$$

Load Transfer = (Mass x Deceleration x Height of centre above ground) / Wheelbase

Calculating final front and rear brake forces

```
#Update front and rear normal loads by transferred
FRONT_LOAD += LOAD_TRANSFER
REAR_LOAD -= LOAD_TRANSFER
```

Using the idea that the rear load transfers force onto the front load, I subtracted the load transfer calculated from the rear load and added that amount onto the front load.

```
#Calculate Front and Rear braking forces
front_brake_force = FRICTION_COEFFICIENT * FRONT_LOAD
rear_brake_force = FRICTION_COEFFICIENT * REAR_LOAD
```

The final step was to adjust the new front and rear brake forces by using F = uR, where F is brake force, u is coefficient of friction and R is reaction force of each axle (the calculated front and rear loads).

I now have the fundamental theory calculations to calculate the front and rear brake forces for a given value of the coefficient of friction

Diagonal reference lines

Ref: https://www.youtube.com/watch?v=iJ7bf20phWc&t=243s

As all points are plotted to create an ideal braking curve there needs to be a diagonal line that is calculated, to visually find the braking forces for a given value of the coefficient of friction on the graph.

The total braking force calculation is:

Front braking force + Rear braking force = coefficient of friction x mass x acceleration due to gravity

F (front) + F (rear) = umg

As the graphs x axis is the front braking force F (front) and the y axis is the rear braking force F(rear), we can swap these variables so that x + y = umg

Hence we can rearrange for y: y = umg -x

Mathematically, if we have two function curves/lines intersect, the point of intersection is where both function curves/lines have the same point. Hence if we go along the diagonal line (at the given value of the coefficient of friction) and check where it intersects with the braking curve, that is the point where BOTH axles reach their grip limits together. Hence we can see the limits of both front and rear together at this value.

Assumptions and decisions

Assumption/Decision	Reason/justification	Effect on model, code or results
Assumed using np.arange would automatically append data as float data into lists.	I thought it was similar to just using in range() so would set values as data goes into it.	When printing the lists as a test, data came out as non formed calculations rather than float data. So I had to add float() to append instead.
Assumed the range for coefficient of friction should be 0<=u<=1	This is the range I chose according to the video linked by the task document.	It resulted in a full final graph with all diagonal lines calculated properly.
Calculated only 10 values of u rather than 11 (due to 0 and 1 being values)	I initially made my diagonal line iteration miss the final line when u = 1 as I plotted too few as I accounted for 0 but not 1.	The final diagonal line where u = 1 was finally displayed correctly.
Using an iteration to plot diagonal lines (y = umg – x) and not parameterise colour as black	I assumed the lines would all default to the colour black	They did not default to black and made the colours of the rainbow which made the model extremely messy and difficult to read.
Using an approximation calculation to plot the values of coefficient of friction beside each of their individual lines.	Given that the real x values of u were slightly lower than the estimation of the total horizontal window, I assumed it would display just next to the line successfully.	Each value of coefficient of friction successfully displayed next to their corresponding lines rather on top.
When defining constant x function using np.linespace(0,2000,200) I assumed that I could leave it as 2000 so long as I used ax.set() with x lim and made it (0,3000) and that would extend the window.	I thought xlim overrides the x function as it was initialised.	It resulted in a larger window, but the model was incomplete as the x values above 2000 did not display, hence I had to extend linspace to 3000.

The road is completely level	For simplicity the road is level so gravity has no component acting along the road.	Made the maths much more simple to graph curve.
Same friction coefficient used for both axles	Made the model simple so that there is only 1 value of u for both axles to make graph simple	Graph is simple to read from as there is only one value of u for both axles
Assume there are no external forces acting such as drag, or weird rotational forces from wind, and neglect environment	Nothing complex is included in calculations and these affects are minimal anyways	Effects not assessed by this model so real world accuracy is limited
Assumed a simple friction model	Allowed to use Fmax = uR simply so no complex detail required	Effects not assessed by this model so real world accuracy is limited

Results and verification

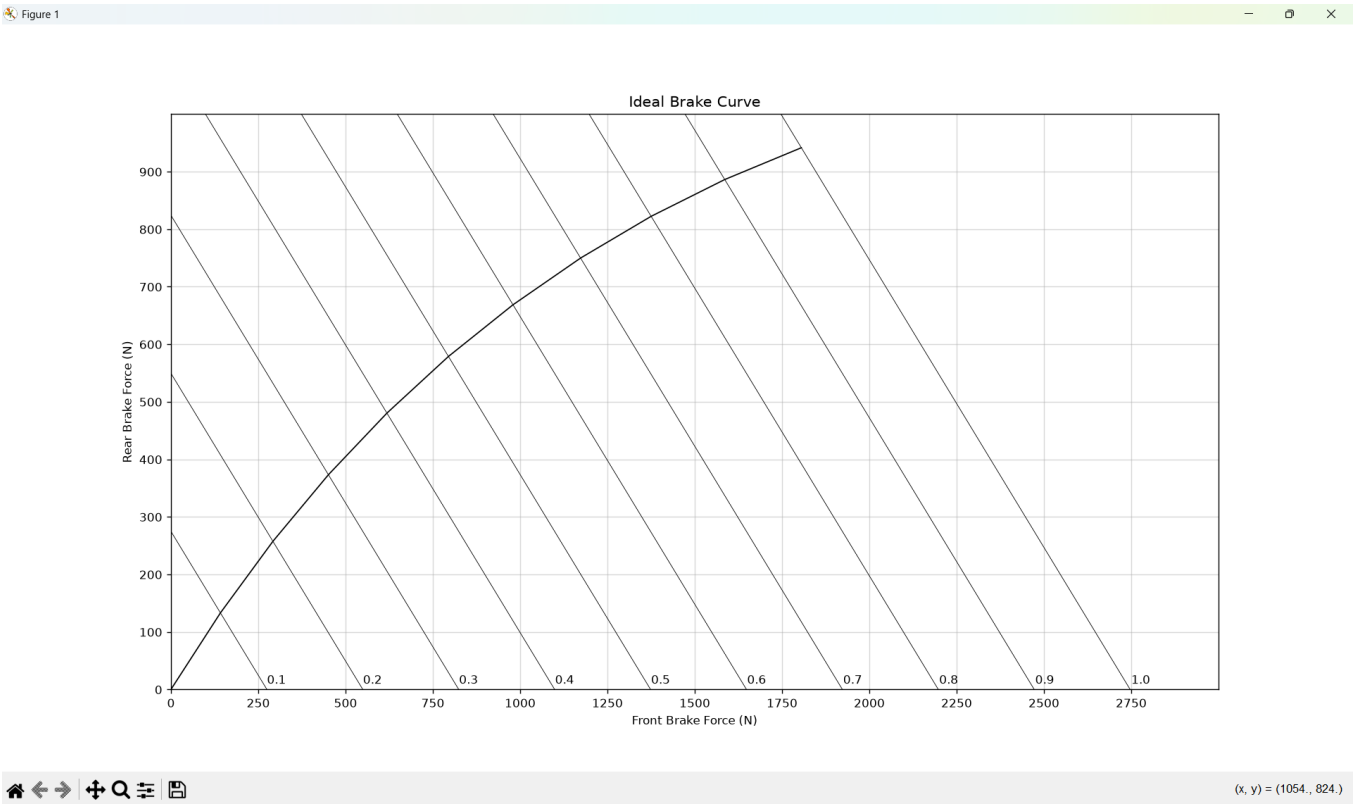


Figure 1, final ideal brake curve displayed from running code

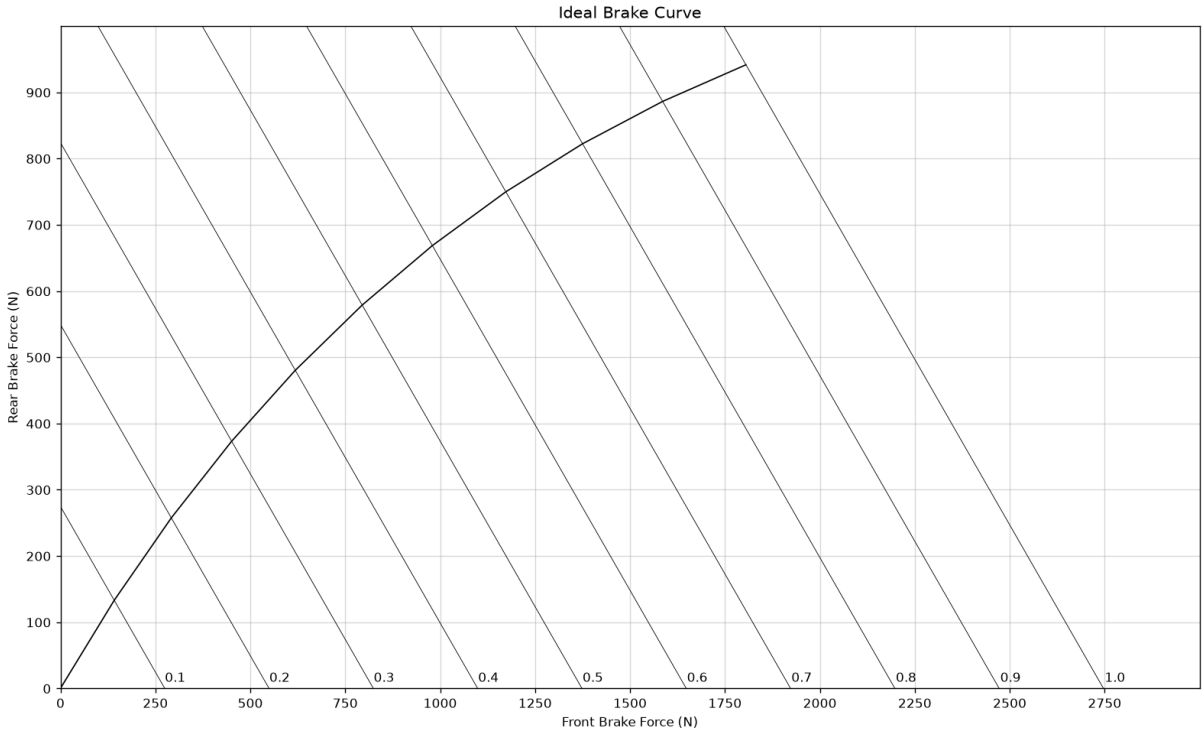


Figure 2, final ideal brake curve zoomed in

How to use

The graph shows an x axis of front brake force, a y axis of rear brake force and diagonal lines of the coefficient of friction values ranging from 0.1 to 1 in intervals of 0.1. To interpret the graph, go to a diagonal of your chosen coefficient of friction value and trace it up to the curve and read the coordinates (x,y) for the (front brake force, rear brake force) pair at that value of u, which is where both reach their grip limits.

Running instructions

(1) Make sure Python is installed along side numpy and matplotlib. To do this, https://www.python.org/downloads/ Click on this link and hover over downloads and then click on your relevant operating system (windows, linux, mac). Then click on download latest version .
(2) Download the numpy and matplot lib packages after python is installed. Open your CMD usually by searching “CMD” into your operating system search bar. Then type in the command “pip install numpy”, once that is done, type in “pip install matplotlib”. This will download these libraries onto python ready to use.
(3) Identify the source code name “QMFS-Software-Senior-Team-Task.py”. Make sure to check it is a py file. Then right click onto the file and click “EDIT WITH IDLE” and open to view code. You can now press F5 to run the code (click okay to save first).
(4) Alternatively if you want to just view the graph and not the code, double click straight onto the py file and the code should run and the graph will display.
(5) In the IDLE once you have viewed the code, if any changes are made and need to be saved, click CTRL and S at the same time to do so.

References

- *Matplotlib.pyplot.plot#* (2026) *matplotlib.pyplot.plot - Matplotlib 3.11.2 documentation*. Available at: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html? (Accessed: 18 October 2026).
- Neurobyte Network (no date) *8. Matplotlib - Plotting Multiple Lines*, *YouTube*. Available at: https://www.youtube.com/watch?v=ry_EVvZVIlY (Accessed: 18 October 2026).

- *Matplotlib.axes.Axes.grid#* (2026) *matplotlib.axes.Axes.grid - Matplotlib 3.11.2 documentation*. Available at: https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.grid.html (Accessed: 19 October 2026).
- Suspension Explained (no date) *The Ideal Brake Curve*, *YouTube*. Available at: https://www.youtube.com/watch?v=iJ7bf20phWc&t=243s (Accessed: 17 October 2026).

Source Code

```
#Import libraries

import matplotlib.pyplot as plt

import numpy as np


#Define constants

GRAVITY_ACCELERATION = 9.81 #m/s^2

MASS = 280 #Kg

COG_HEIGHT = 0.25 #centre of gravity height #metres

WHEELBASE = 1.59 #metres

FRONT_WEIGHT_DISTRIBUTION = 0.5


#predefine lists

front_brake_list =[]
```

```
rear_brake_list = []

const_frict_list = []


#define constant x function for Friction coefficient lines (these will read the braking values)

x = np.linspace(0,3000,200)



#Loop using each value of u to calculate data points using braking curve theory

for num in np.arange(0, 1.1, 0.1):


  #define and reset front and rear loads

  FRONT_LOAD = MASS * GRAVITY_ACCELERATION * FRONT_WEIGHT_DISTRIBUTION

  REAR_LOAD = (MASS * GRAVITY_ACCELERATION) * (1 - FRONT_WEIGHT_DISTRIBUTION)


  #friction coefficient 0<u<1

  FRICTION_COEFFICIENT = num


  #Deceleration calculation  a = ug                    #calculate deceleration

  DECELERATION = FRICTION_COEFFICIENT * GRAVITY_ACCELERATION


  #Transferred load from rear to front calculation          #use this value to calculate load transfer

  LOAD_TRANSFER = (MASS * DECELERATION * COG_HEIGHT) / WHEELBASE  #F(transfer) = ma D(COG) / D(wheelbase)


  #Update front and rear normal loads by transferred

  FRONT_LOAD += LOAD_TRANSFER                    #add load transfer to front load as it gains from rear

  REAR_LOAD -= LOAD_TRANSFER                     #subtract load from rear as it givse load to front


  #Calculate Front and Rear braking forces

  front_brake_force = FRICTION_COEFFICIENT * FRONT_LOAD      #use F = uR, To calculate brake forces, where R is the Normal force of each axle (front and rear loads)

  rear_brake_force = FRICTION_COEFFICIENT * REAR_LOAD       #using values of the loads for each axle
```

```
    #Add brake forces to list
    front_brake_list.append(float(front_brake_force))
    rear_brake_list.append(float(rear_brake_force))

    #Create line function y = umg - x
    y = (FRICTION_COEFFICIENT * MASS * GRAVITY_ACCELERATION) - x    #y = umg - x   is used to create diagnol line at each friction coefficient point
    #append to line list
    const_frict_list.append(y)



#Plotting code



#create and adjust axes
fig, ax = plt.subplots(figsize=(25,10))     #sets display axis 25 inches horizontally, 10 inches vertically
#make grid
ax.grid(alpha=0.5)     #sets a grid structure where alpha halves opacity to make grid feint

#plot data

#brake curve
ax.plot(front_brake_list, rear_brake_list, linewidth = 1.0, color = 'black')   #plots curve on axis, linewidth 1px, black color

#y = umg - x
for count in np.arange(0,11):                                #This code iterates through each friction coefficient value and
    ax.plot(x, const_frict_list[count], color = 'black', linewidth = 0.5)   #it plots the lines y = umg - x  for each value
```

```
#label each point with coefficient number

for label in np.arange(0.1, 1.1, 0.1):

   ax.text( (2750 * label) ,10, f'{round(label,1)}')      #mathematically approximate label points and place values to show them



#adjust graph and name axes

ax.set(xlim=(0,3000), xticks = np.arange(0,3000,250),   #xlim sets limit of x axis 0 - 3000, ticks display intervals every 250 units

    ylim=(0,1000), yticks = np.arange(0,1000,100),   #ylim sets limit of y axis 0 - 1000, ticks display intervals every 100 units

    xlabel='Front Brake Force (N)', #x label

    ylabel='Rear Brake Force (N)', #y label

    title='Ideal Brake Curve'          ) #title


#show plots

plt.show()
```